

Parking Manager Project Guide

1. What this project does

Parking Manager is a small web app for running a 50-space car park. It records vehicle arrivals, assigns bays named G1 to G50, calculates parking fees, starts M-Pesa payments through Safaricom Daraja, and records daily revenue.

The app can print an entry ticket and a payment receipt using the browser's print window. The barrier is simulated in the software. A real gate needs a separate gate controller and hardware connection.

2. Main features

- Shows all 50 bays and marks each one available or occupied.
- Assigns the first available bay when a vehicle arrives.
- Prints a ticket with a ticket number, plate, bay, and arrival time.
- Calculates parking time and fee when a vehicle exits.
- Sends an M-Pesa STK Push request after the driver enters a phone number and presses **Enter**.
- Waits for payment confirmation before preparing a receipt.
- Keeps a bay occupied until the receipt is printed and the exit action completes.
- Shows payment totals and exit counts for the current day.

3. Project files

- `module1_display.py`: Starts the Flask web app and provides the dashboard, bay list, parked-vehicle list, and health check.
- `module2_entry.py`: Checks a vehicle in, assigns a bay, and returns entry-ticket details.
- `module3_exit.py`: Finds the vehicle, calculates time and fee, and creates a pending exit.
- `module4_payment.py`: Starts Daraja STK Push, checks payment status, handles callbacks, creates receipt details, and completes the exit.
- `module5_database.py`: Opens the SQLite database and safely adds new columns when the app starts.
- `module6_management.py`: Calculates today's payment total and exit count.
- `schema.sql`: Defines the initial database tables and adds 50 available bays when a new database is created.
- `templates/index.html`: Contains the dashboard, browser interactions, ticket and receipt layouts, and print styling.
- `check_daraja_setup.py`: Checks Daraja settings, the callback address, OAuth access, and optionally sends an STK test.
- `.env.example`: Safe template for local settings. It contains placeholders, not private keys.
- `.env`: Private local settings. It is ignored by Git and must not be shared or committed.
- `requirements.txt`: Lists the Python packages needed by the app.
- `modern_parking.db`: Local SQLite data file. It is created when the app starts for the first time.
- `PROJECT_DOCUMENTATION.md`: This guide.

The workspace also includes `Algorithm.txt` and `Data_structures.txt`. These are assignment notes describing the planned modules and data structures. The running app uses SQLite tables instead of separate in-memory sets, maps, or queues.

4. How vehicle entry works

1. Enter the vehicle's plate and choose **Simulate Entry & Open Barrier**.
2. The app checks that the plate is not already parked and looks for an available bay.
3. The app changes that bay to occupied and saves a parking session with the arrival time.
4. The app displays an entry ticket and opens the browser print window.
5. The ticket shows its number, plate, assigned bay, and arrival time.

If no bays are available, the app reports that the car park is full. The entry action is a software simulation; it does not control a physical barrier.

5. How exit and fees work

Enter the same plate in **Exit & Billing** and choose **Calculate Fee**. The app finds the current session, saves the exit time, and displays the time parked and the fee. The fee is kept with the session so that repeating the request does not recalculate it.

Fee rules use the total time rounded up to whole minutes:

- Up to 30 minutes: KES 0.
- 31 minutes to 2 hours: KES 50.
- More than 2 hours and up to 4 hours: KES 100.
- More than 4 hours and up to 6 hours: KES 300.
- More than 6 hours: KES 500.

For a fee above zero, enter a 10-digit Kenyan number beginning with 01 or 07, then press **Enter** beside the phone field. Typing alone does not send a payment request. A zero-fee exit does not require an M-Pesa prompt, but still produces a receipt before the bay is freed.

6. How payment, receipt, and exit work

1. The browser sends the plate and phone number to the payment route.
2. The server validates the number and asks Daraja to start an STK Push.
3. The driver approves the request on the phone when using a live M-Pesa account.
4. Safaricom sends a callback with the result. The app marks payment as paid only when the result is successful and the amount matches the fee.
5. If the callback is late or lost, the status route can query Daraja after a short wait. Queries are throttled so the app does not ask too often.
6. The app displays receipt details, including the receipt reference, plate, bay, parked time, amount, masked phone, and payment time.
7. Choose **Print Receipt & Open Barrier**. The browser print window opens; after that action, the app completes the session and marks the bay available.

The app does not free the bay when an STK request is merely accepted. A request being accepted is not the same as payment being completed.

7. M-Pesa sandbox and production

The current setup uses the Daraja **sandbox**. Sandbox requests are simulated and do not send a prompt to a normal personal phone or move money. The recognised test number is 254708374149; enter its 10-digit local form, 0708374149, in this app. The dashboard shows a warning when sandbox is active, and the server rejects other numbers before contacting Daraja.

To receive a prompt on a real phone, the business needs an approved production Daraja app, its own Paybill or Till, production consumer key and secret, and the matching production passkey. Set `DARAJA_ENV=production` only after Safaricom approves the account. Real production requests can move money.

The credentials previously shared in chat should be revoked and replaced. Never put private consumer keys or secrets in this guide, `.env.example`, or source code. Store them only in the ignored `.env` file or another approved secret store.

8. Database tables

Parking_Slots

Stores the 50 bays. `SlotID` is the number in SQLite; the screen displays it with a G prefix. Status is Available or Occupied.

Parking_Sessions

Stores one parking visit: plate, bay, arrival and exit times, amount due, session status, payment status, phone used for the request, Daraja request IDs, and M-Pesa receipt number.

Session status is normally:

Active -> Pending_Payment -> Completed

Payment status is normally:

NotStarted -> Initiating -> Requested -> Paid Or Failed

Payment_Logs

Stores completed payment records, including the session, masked-at-display phone information, amount, date, and Daraja receipt number when available.

When an older database is opened, `init_db()` adds missing payment columns. It does not recreate an existing database. Do not run the `DROP TABLE` statements in `schema.sql` against data you want to keep.

9. Web routes

- GET `/`: Opens the dashboard.
- GET `/healthz`: Reports whether the app is running, the Daraja environment, and available bay count.
- GET `/api/slots`: Returns every bay and its status.
- GET `/api/active-vehicles`: Returns parked and payment-pending vehicles.
- POST `/api/entry`: Creates an entry session and returns ticket data. JSON body: `{"plate": "KCA123A"}`.
- POST `/api/exit`: Calculates or recovers the fee for a plate. JSON body: `{"plate": "KCA123A"}`.
- POST `/api/pay/mpesa`: Requests payment. JSON body includes `plate` and a 10-digit local phone.
- GET `/api/pay/mpesa/status?plate=KCA123A`: Returns payment state and receipt details after payment.
- POST `/api/pay/mpesa/callback`: Receives Safaricom Daraja's payment result. This address must be reachable from the internet over HTTPS.
- POST `/api/exit/complete`: Completes a paid exit after the receipt-print step and frees the bay.
- GET `/api/revenue`: Returns today's payment total and number of payment-log entries.

10. Run the app on Windows

Open PowerShell in the `Parking_System` folder and run:

```
python -m pip install -r requirements.txt
Copy-Item .env.example .env
python module1_display.py
```

Open `http://127.0.0.1:5000` in a browser. If `.env` already exists, do not overwrite it with the copy command. Edit that private file locally and keep its credentials secret.

The app uses Flask's development server for local work. Use a production WSGI server and HTTPS before accepting real payments.

11. Daraja setup and checks

The private `.env` file needs these settings:

```
DARAJA_ENV=sandbox
DARAJA_CONSUMER_KEY=your_key
DARAJA_CONSUMER_SECRET=your_secret
DARAJA_SHORTCODE=174379
DARAJA_PASSKEY=your_matching_passkey
DARAJA_CALLBACK_URL=https://your-public-host/api/pay/mpesa/callback
DARAJA_TRANSACTION_TYPE=CustomerPayBillOnline
DARAJA_ACCOUNT_REFERENCE=Parking
```

The sandbox shortcode and passkey are shared test values. In production, use the shortcode and passkey issued for the business account. `CustomerPayBillOnline` is for a Paybill; `CustomerBuyGoodsOnline` is for a Till.

Daraja cannot call `localhost`. For sandbox testing, expose port 5000 with an HTTPS tunnel, then set `DARAJA_CALLBACK_URL` to the tunnel address ending in `/api/pay/mpesa/callback`. Restart the app after changing `.env`. A free tunnel address can change when the tunnel restarts.

Run the safe configuration check with:

```
python check_daraja_setup.py
```

That checks settings, callback reachability, and OAuth but does not send an STK Push unless `--stk` is provided. Treat `python check_daraja_setup.py --stk PHONE AMOUNT` as a payment action: in production it can send a real request and move money if approved.

12. Printing and barrier hardware

The app formats tickets and receipts for an 80 mm paper width and uses the browser's print window. Choose the installed thermal printer in that window. The software currently simulates opening the barrier; it does not connect to a gate motor, relay, sensor, or access-control device.

13. Troubleshooting

- **No real-phone prompt:** Check `/healthz`. In sandbox, use only 0708374149; sandbox will not contact your personal number. Real prompts need production approval and credentials.
- **"No pending payment found":** Calculate the fee first for a plate with an active parking session.
- **Invalid phone message:** Enter exactly 10 digits beginning with 01 or 07, with no country prefix or spaces.
- **Payment says requested but not paid:** The driver may still need to approve the prompt. If the callback is missing, the app queries Daraja after a short wait. Check that the HTTPS tunnel is running and the callback URL has not changed.
- **Immediate retry is refused:** Wait for the retry cooldown. M-Pesa briefly locks a number after an STK request.
- **Payment accepted but bay remains occupied:** This is expected until the receipt is printed and the exit action completes.
- **App will not start:** Install `requirements.txt` dependencies and confirm port 5000 is free.
- **Database appears inconsistent:** Keep a backup before making manual changes. The dashboard displays the stored slot and session data; avoid resetting the database just to correct one bay.

14. Tests and checks

The project was checked with Python compilation, isolated SQLite tests for entry, fee, payment callback, receipt and exit, and browser tests for the ticket, Enter button, phone validation, and receipt layout. Daraja sandbox acceptance is not proof that a real phone was contacted. A production test must be done only with Safaricom-approved live settings and a phone you are allowed to test.